ARQUITECTURA DE SISTEMA DE BANCA DIGITAL BP

Documento técnico

Diseño banca digital para soporte de operaciones críticas como consultas de movimientos, transferencias y pagos

Gabriel Huayamabe Rizo

Arquitectura de Sistema de Banca Digital BP

Autor: Gabriel Huayamabe

Rol: Arquitecto de Soluciones

Fecha: 4 de abril del 2026

Contenido

Introducción 4
Requerimientos 5
Requerimientos funcionales 5
Requerimientos no funcionales 5
Comparación de Tecnologías 6
FrontEnd Web 6
Justificación: 6
Trade-off: 6
Alternativa evaluada: 6
Conclusión técnica para React (Frontend Web) 7
Aplicación Móvil 7
Justificación técnica 7
Compilación AOT (Ahead-of-Time) a código nativo 7
Eliminación del bridge (menor overhead) 8
Mejor desempeño en UI complejas y animaciones 8
Consistencia en dispositivos de gama media y baja 8
Trade-offs 8
Alternativa evaluada: React Native 8
Motivos de descarte 8
Conclusión técnica 9
Cloud 9
Justificación técnica 9
Amplio portafolio de servicios administrados 9
Alta disponibilidad mediante despliegue Multi-AZ 9
Escalabilidad automática con Auto Scaling 10
Integración nativa con arquitecturas distribuidas 10
Seguridad integrada y cumplimiento 10
Trade-offs 10
Alternativa evaluada: Azure 10
Motivos de descarte 10
Conclusión técnica 11
Arquitectura General 11

Actores principales 11
Flujo principal 11
Consideración adicional 12
Arquitectura de Contenedores 12
Componentes principales: 12
Arquitectura de Componentes 12
Se definen los siguientes microservicios: 12
Patrones utilizados: 12
Protocolos: 12
Autenticación 13
Onboarding y Biometría 13
Integración 14
Auditoría 14
Cache 15
Notificaciones 15
Alta Disponibilidad y Resiliencia 16
Monitoreo 16
Regulaciones 16
Manejo de Costos 17
Diagrama de Contexto 18
Diagrama de Contenedores 19
Diagrama de componentes 19
Diagrama Flujo Login 20
Diagrama Flujo transferencia 21

Introducción

Este documento describe la arquitectura de un sistema de banca digital diseñado para soportar operaciones críticas como consulta de movimientos, transferencias y pagos entre cuentas propias e interbancarias.

La solución está basada en principios de arquitectura moderna como microservicios, desacoplamiento, alta disponibilidad, resiliencia, seguridad y escalabilidad en la nube. Además, se incorporan decisiones arquitectónicas justificadas, análisis de trade-off y consideraciones basadas en errores comunes que suelen presentarse en sistemas distribuidos reales.

El objetivo es diseñar una arquitectura robusta, mantenible y preparada para crecimiento futuro, cumpliendo con requerimientos funcionales, no funcionales y regulatorios del sector financiero.

Requerimientos

Requerimientos funcionales

- Consulta de movimientos históricos
- Transferencias entre cuentas propias
- Transferencias interbancarias
- Pagos de servicios
- Notificación de transacciones
- Onboarding de clientes con reconocimiento facial
- Autenticación mediante OAuth 2.0
- Acceso a información desde sistemas Core y sistemas complementarios

Requerimientos no funcionales

- Alta disponibilidad (HA)
- Tolerancia a fallos
- Baja latencia
- Escalabilidad horizontal
- Seguridad (OAuth2, TLS, cifrado de datos en tránsito y en reposo)
- Auditoría completa de operaciones
- Monitoreo y observabilidad
- Recuperación ante desastres (DR)
- Desacoplamiento de componentes

Comparación de Tecnologías

FrontEnd Web

Se selecciona React para la aplicación SPA

Justificación:

1. Modelo de renderizado eficiente (Virtual DOM y reconciliación), React implementa un Virtual DOM que permite realizar un proceso de diffing antes de actualizar el DOM real. Esto reduce operaciones costosas y mejora el rendimiento en interfaces con alta interacción, como dashboard financieros y consultas dinámicas.
2. Arquitectura basada en componentes desacoplados, React promueve componentes reutilizables alineados con principios de separación de responsabilidades. Esto facilita la escalabilidad del FrontEnd y reduce el acoplamiento.
3. Manejo avanzado de estado y datos asincrónicos, El ecosistema permite integrar herramientas como Redux o React Query, habilitando cache en FrontEnd, sincronización eficiente con APIs y reducción de llamadas innecesarias al BackEnd.
4. Integración con arquitecturas API-first, React se adapta fácilmente a arquitecturas basadas en API Gateway y microservicios, permitiendo desacoplar completamente FrontEnd y BackEnd.
5. Optimización de performance, permite implementar lazy loading, code splitting y optimización de bundle, mejorando tiempos de carga

Trade-off:

1. Requiere definir estándares internos (estructura, estado, routing)
2. Mayor responsabilidad en decisiones arquitectónicas frontend

Alternativa evaluada:

Angular

Motivos de descarte:

1. Mayor rigidez arquitectónica
2. Mayor tamaño inicial del bundle
3. Curva de aprendizaje más alta
4. Uso intensivo de RxJS que incrementa complejidad

Conclusión técnica para React (Frontend Web)

React se consolida como una opción adecuada para el frontend web debido a su flexibilidad arquitectónica, su modelo eficiente de renderizado mediante Virtual DOM y su capacidad de integrarse de forma natural con arquitecturas basadas en microservicios y API Gateway.

Permite optimizar la experiencia del usuario en interfaces altamente dinámicas, gestionar de forma eficiente el estado y los datos asincrónicos, y adaptarse a distintos requerimientos del negocio sin imponer restricciones estructurales rígidas.

Aunque requiere definir estándares internos para mantener consistencia, esta flexibilidad ofrece mayor control sobre la evolución del sistema, lo que resulta especialmente valioso en entornos como la banca digital donde los requerimientos cambian constantemente y la performance es un factor crítico.

Aplicación Móvil

Justificación técnica

Arquitectura de renderizado propia (sin dependencia de componentes nativos)

Flutter utiliza su propio motor de renderizado (Impeller), lo que significa que no depende de componentes nativos del sistema operativo.

Esto permite:

1. UI consistente en iOS y Android
2. Control total sobre el renderizado
3. Menor variabilidad entre dispositivos

A diferencia de otras soluciones, no existe traducción entre capas de UI, lo que reduce inconsistencias.

Compilación AOT (Ahead-of-Time) a código nativo

Flutter compila código Dart directamente a código máquina (ARM), eliminando la necesidad de un runtime adicional.

Esto implica:

1. Mejor tiempo de ejecución
2. Menor latencia en operaciones críticas
3. Mayor estabilidad en producción

Esto lo hace más predecible en escenarios exigentes como sistemas financieros.

Eliminación del bridge (menor overhead)

Flutter no requiere una capa intermedia entre lógica y UI.

Esto evita:

1. Latencia en la comunicación
2. Cuellos de botella
3. Overhead por serialización de datos

La ejecución es directa sobre el motor gráfico.

Mejor desempeño en UI complejas y animaciones

Flutter mantiene tasas de refresco estables incluso en interfaces complejas.

Esto es clave para:

1. Dashboard financieros
2. Visualización de datos
3. Transiciones complejas

Consistencia en dispositivos de gama media y baja

Flutter mantiene un comportamiento estable en dispositivos con recursos limitados.

Esto es relevante en entornos reales donde los usuarios no siempre cuentan con hardware de alto rendimiento.

Trade-offs

1. Mayor tamaño inicial de la aplicación
2. Necesidad de aprender Dart
3. Ecosistema en crecimiento comparado con tecnologías más antiguas

Alternativa evaluada: React Native

Motivos de descarte

Dependencia de una capa intermedia entre JavaScript y código nativo, lo que puede introducir latencia y complejidad adicional en la comunicación.

El rendimiento puede depender del tipo de aplicación, requiriendo optimizaciones adicionales o desarrollo de módulos nativos en escenarios complejos.

Uso de componentes nativos que puede generar diferencias de comportamiento entre plataformas.

Conclusión técnica

Flutter se selecciona por ofrecer mayor control sobre el renderizado, un rendimiento más predecible y menor complejidad en escenarios críticos, lo que lo hace adecuado para aplicaciones donde la estabilidad y consistencia son prioritarias.

Cloud

Tecnología seleccionada: AWS

Justificación técnica

Amplio portafolio de servicios administrados

AWS ofrece un ecosistema completo de servicios gestionados que cubren cómputo, almacenamiento, bases de datos, mensajería, monitoreo y seguridad.

Esto permite:

- Reducir la carga operativa del equipo
- Enfocarse en lógica de negocio en lugar de infraestructura
- Implementar rápidamente patrones arquitectónicos como microservicios, event-driven y serverless

Alta disponibilidad mediante despliegue Multi-AZ

AWS permite desplegar servicios en múltiples zonas de disponibilidad dentro de una misma región.

Esto garantiza:

- Tolerancia a fallos a nivel de infraestructura
- Continuidad del servicio ante caídas parciales
- Distribución del tráfico mediante balanceadores de carga

Este enfoque es fundamental en sistemas bancarios donde la disponibilidad es crítica.

Escalabilidad automática con Auto Scaling

AWS permite escalar horizontalmente servicios en función de la demanda mediante políticas de Auto Scaling.

Esto permite:

- Adaptar la capacidad del sistema en tiempo real
- Optimizar costos al no sobredimensionar recursos
- Soportar picos de carga sin degradación del servicio

Integración nativa con arquitecturas distribuidas

AWS proporciona servicios que facilitan la implementación de arquitecturas desacopladas:

- API Gateway para exposición de servicios
- Servicios de mensajería para comunicación asíncrona
- Bases de datos administradas y altamente disponibles
- Herramientas de monitoreo y observabilidad integradas

Esto permite construir soluciones alineadas con principios de microservicios y event-driven architecture.

Seguridad integrada y cumplimiento

AWS incorpora mecanismos de seguridad a nivel de infraestructura y servicios:

- Gestión de identidades y accesos (IAM)
- Cifrado de datos en tránsito y en reposo
- Control de redes mediante VPC

Además, cuenta con certificaciones y cumplimiento de estándares relevantes para el sector financiero, lo que facilita alinearse con regulaciones.

Trade-offs

- Complejidad inicial en la configuración de servicios
- Riesgo de dependencia del proveedor (vendor lock-in)
- Necesidad de control y monitoreo de costos para evitar sobreuso

Alternativa evaluada: Azure

Motivos de descarte

Menor alineación con las herramientas y experiencia actual del equipo, lo que puede impactar tiempos de implementación y curva de aprendizaje.

Aunque Azure es una plataforma robusta, en este caso específico se prioriza la madurez y adopción de ciertos servicios de AWS en arquitecturas distribuidas, así como la familiaridad del equipo con su ecosistema.

Conclusión técnica

AWS se selecciona por su capacidad de soportar arquitecturas distribuidas altamente disponibles, su amplio portafolio de servicios administrados y su flexibilidad para escalar según la demanda.

Su enfoque en automatización, resiliencia y seguridad lo convierte en una opción adecuada para sistemas bancarios donde la continuidad del servicio, la protección de datos y la capacidad de adaptación a la carga son factores críticos.

Arquitectura General

La arquitectura se basa en un enfoque de microservicios desacoplados, expuestos a través de un API Gateway, lo que permite separar responsabilidades y facilitar la evolución independiente de cada componente del sistema.

Actores principales

- Usuario Web (SPA)
- Usuario Mobile
- Sistema Core Bancario
- Sistema de datos complementarios
- Servicios de notificaciones

Cada uno de estos actores interactúa mediante interfaces bien definidas, evitando dependencias directas y permitiendo una integración controlada y escalable.

Flujo principal

El flujo inicia en los clientes (web o móvil), que realizan solicitudes a través de APIs seguras. Estas solicitudes son recibidas por el API Gateway, que actúa como punto único de entrada y se encarga de validar autenticación, aplicar políticas de seguridad y enrutar la petición al microservicio correspondiente según el tipo de operación.

Los microservicios procesan la lógica de negocio y, cuando es necesario, consumen información desde el sistema Core Bancario o el sistema de datos complementarios, asegurando que los datos se obtengan de las fuentes oficiales.

Para operaciones como notificaciones, el sistema puede apoyarse en mecanismos asíncronos, desacoplando estos procesos del flujo principal para no afectar los tiempos de respuesta al usuario.

Consideración adicional

Este enfoque permite mejorar la escalabilidad y resiliencia del sistema, ya que cada servicio puede evolucionar, escalar o fallar de manera independiente sin comprometer el funcionamiento global de la plataforma.

Arquitectura de Contenedores

Componentes principales:

- SPA (React)
- Aplicación móvil (Flutter)
- API Gateway
- Servicio de autenticación
- Servicios de negocio (cliente, movimientos, transferencias)
- Servicio de notificaciones
- Base de datos
- Sistema de auditoría
- Cache (Redis)

Cada contenedor cumple una responsabilidad específica y se comunica mediante APIs REST o mensajería asíncrona.

Arquitectura de Componentes

Se definen los siguientes microservicios:

- Cliente Service
- Movimientos Service
- Transferencias Service
- Notificaciones Service
- Auth Service

Patrones utilizados:

- API Gateway Pattern
- Microservices Pattern
- Circuit Breaker
- Retry Pattern
- Event-driven Architecture

Protocolos:

- HTTP/REST
- Mensajería (eventos para notificaciones)

Autenticación

Se utiliza OAuth 2.0 con flujo Authorization Code + PKCE.

Justificación:

- Flujo seguro para aplicaciones públicas
- Evita exposición de credenciales
- Compatible con SPA y mobile
- No requiere almacenamiento de client secret en el cliente
- Protege contra interceptación del authorization code mediante validación dinámica (PKCE)
- Permite delegar la autenticación en un proveedor especializado (Identity Provider)

Consideración técnica adicional:

PKCE (Proof Key for Code Exchange) introduce un mecanismo en el cual el cliente genera un valor dinámico (code verifier) que es transformado en un code challenge enviado durante la autenticación. Posteriormente, este valor es validado por el servidor al momento de intercambiar el authorization code por el token, asegurando que solo el cliente original pueda completar el flujo.

Errores comunes evitados:

- Almacenamiento inseguro de tokens
- Tokens sin expiración controlada
- Uso de flujos inseguros como Implicit Flow
- Exposición de credenciales en el frontend
- Falta de validación del origen de la autenticación

Onboarding y Biometría

El Onboarding incluye reconocimiento facial.

Justificación:

- Mejora la experiencia del usuario al reducir fricción en el registro
- Incrementa seguridad en el proceso de alta de clientes
- Permite validación de identidad basada en características biométricas únicas, reduciendo fraude de suplantación
- Facilita cumplimiento de procesos KYC (Know Your Customer) requeridos en banca

Se complementa con validaciones BackEnd para evitar falsos positivos, incluyendo validación contra fuentes oficiales y controles antifraude.

Error común:

- Confiar únicamente en biometría sin validación adicional

Integración

El sistema se integra con:

- Core bancario
- Sistema de datos complementarios

Se implementa mediante API Gateway y servicios específicos.

Justificación:

- Reutilización de sistemas existentes evitando duplicidad de lógica
- Separación de responsabilidades entre sistemas legacy y nuevos servicios
- Encapsulamiento del Core para evitar exposición directa
- Permite evolucionar la arquitectura sin afectar sistemas críticos

Error común:

- Alta dependencia del Core que impacta latencia

Mitigación:

- Uso de cache
- Estrategias de desacoplamiento (por ejemplo, consumo asíncrono o replicación de datos)

Auditoría

Se implementa una base de datos de auditoría centralizada.

Justificación:

- Cumplimiento normativo (trazabilidad obligatoria en sistemas financieros)
- Registro completo de acciones del usuario para análisis forense
- Soporte para monitoreo de fraude y revisiones regulatorias
- Permite reconstruir eventos ante incidentes

Se utiliza logging estructurado con identificadores únicos de transacción para correlación entre servicios.

Error común:

- Logs no estructurados o incompletos

Cache

Se utiliza Redis con patrón Cache Aside.

Justificación:

- Reducción significativa de latencia en consultas frecuentes
- Disminución de carga sobre sistemas Core, que suelen ser más lentos
- Mejora de la experiencia del usuario en operaciones de lectura

Error común:

- Mala invalidación que genera inconsistencias

Notificaciones

Arquitectura basada en eventos.

Canales:

- Email
- SMS / Push

Justificación:

- Desacoplamiento del flujo principal mediante procesamiento asíncrono
- Mejora del rendimiento al no bloquear transacciones críticas
- Escalabilidad independiente del sistema de notificaciones
- Garantía de entrega mediante colas o reintentos

Error común:

- Procesos síncronos que afectan la experiencia del usuario

Alta Disponibilidad y Resiliencia

Se implementan:

- Balanceadores de carga
- Multi-AZ
- Circuit Breaker
- Retry Pattern

Justificación:

- Continuidad del servicio ante fallos parciales
- Aislamiento de errores para evitar fallos en cascada
- Recuperación automática ante errores transitorios
- Mejora de la confiabilidad del sistema

Error común:

- No probar escenarios reales de fallo

Monitoreo

Se implementa:

- Logs centralizados
- Métricas
- Alertas

Justificación:

- Observabilidad completa del sistema distribuido
- Detección temprana de fallos y cuellos de botella
- Capacidad de análisis de comportamiento del sistema
- Soporte para troubleshooting en producción

Error común:

- Alertas sin contexto o no accionables

Regulaciones

Se consideran:

- Protección de datos personales
- Cifrado de información
- Cumplimiento PCI DSS

Justificación:

- Protección de información sensible del cliente
- Cumplimiento de normativas legales y regulatorias
- Reducción de riesgos legales y reputacionales
- Garantía de integridad y confidencialidad de los datos

Manejo de Costos

El manejo de costos se enfoca en optimizar el uso de los recursos en la infraestructura Cloud, asegurando que el sistema opere de manera eficiente sin generar gastos innecesarios, manteniendo un equilibrio entre rendimiento, disponibilidad y costo operativo.

Justificación:

Permite controlar el gasto en una arquitectura distribuida donde múltiples servicios consumen recursos de forma independiente, evitando sobrecostos derivados de un uso ineficiente o falta de monitoreo.

Facilita la toma de decisiones basada en consumo real, permitiendo ajustar la infraestructura de acuerdo con la demanda del sistema.

Es clave en entornos Cloud bajo el modelo de pago por uso, donde cada recurso activo genera costos continuos.

Estrategias implementadas:

Se aplica dimensionamiento adecuado de recursos (right-sizing), evitando sobredimensionar instancias o servicios.

Se configuran políticas de Auto Scaling con límites definidos para controlar el crecimiento automático de recursos ante picos de demanda.

Se utilizan servicios administrados que reducen costos operativos asociados a mantenimiento, soporte e infraestructura.

Se implementa cache para reducir el consumo de servicios críticos como bases de datos y sistemas Core, disminuyendo costos de procesamiento y latencia.

Se establecen prácticas para apagar o reducir recursos en ambientes no productivos fuera de horarios de uso.

Se implementa monitoreo continuo del consumo y generación de alertas para detectar incrementos inusuales en el gasto.

Error común:

No definir límites en el escalado automático, lo que puede generar incrementos significativos en costos.

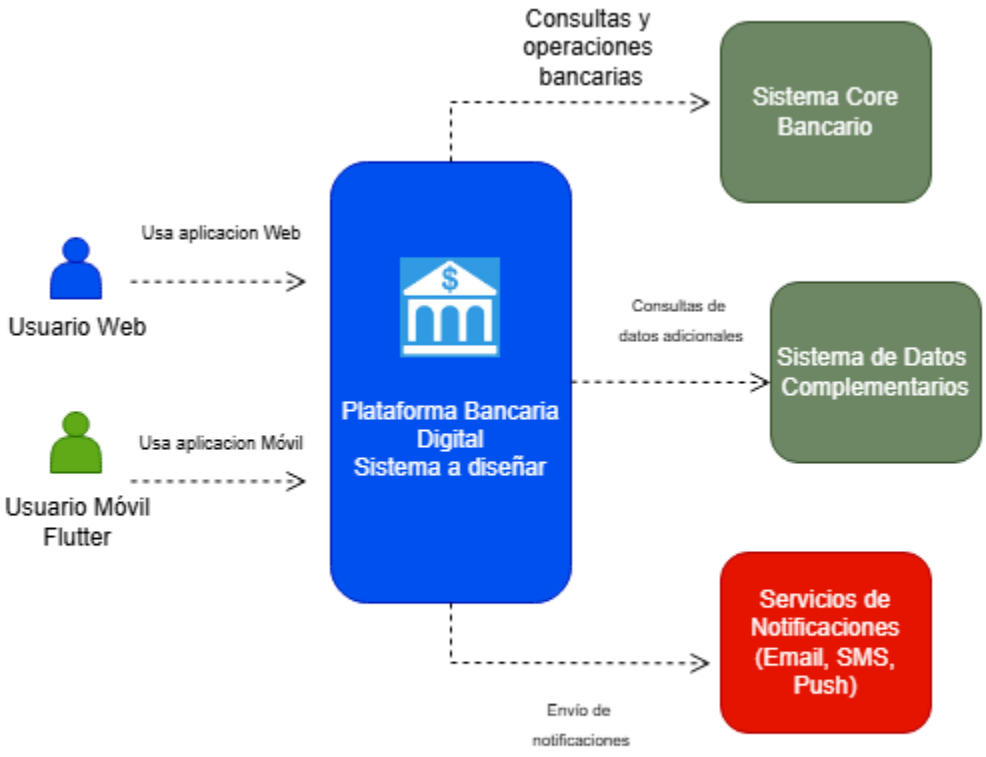
No monitorear el consumo de recursos, lo que impide identificar ineficiencias.

Mantener recursos activos sin uso, generando gasto innecesario.

Conclusión técnica:

El manejo de costos es un componente esencial en arquitecturas Cloud modernas, ya que permite garantizar sostenibilidad financiera del sistema sin comprometer su rendimiento ni disponibilidad, alineando el uso de recursos con la demanda real del negocio.

Diagrama de Contexto



El sistema centraliza el acceso de los usuarios y se integra con sistemas externos a través de APIS seguras

Diagrama de Contenedores

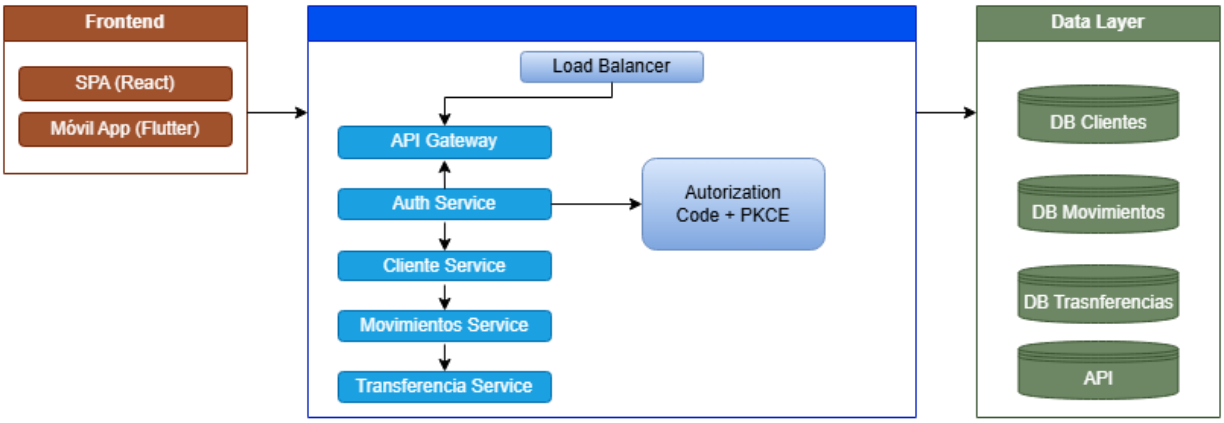


Diagrama de componentes

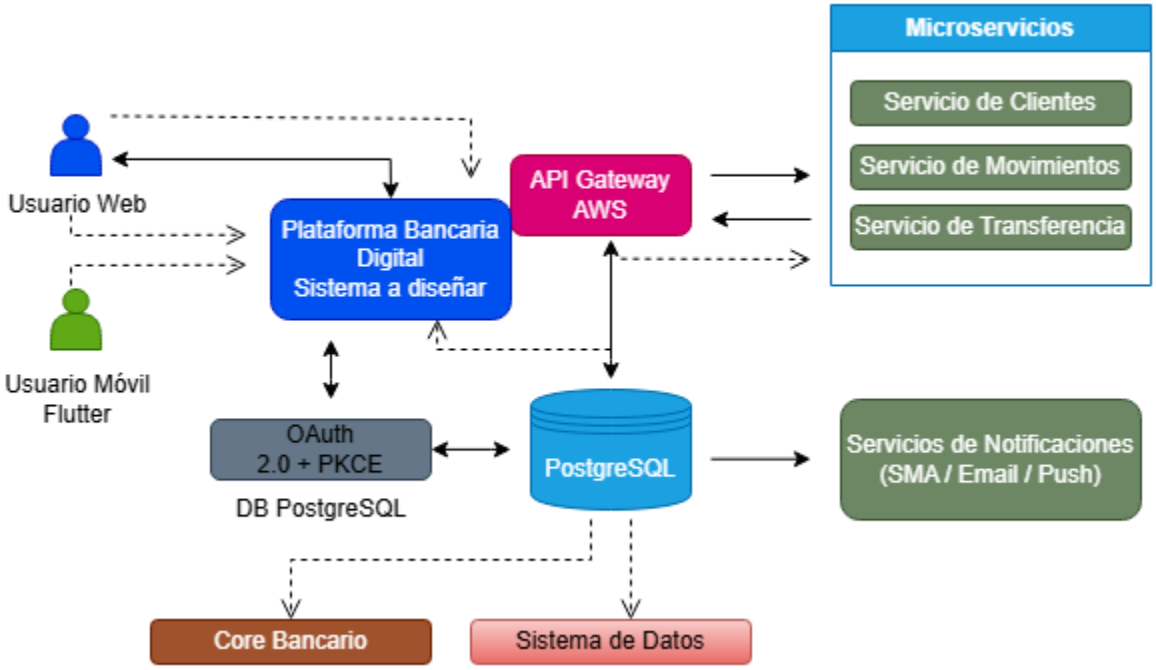


Diagrama Flujo Login

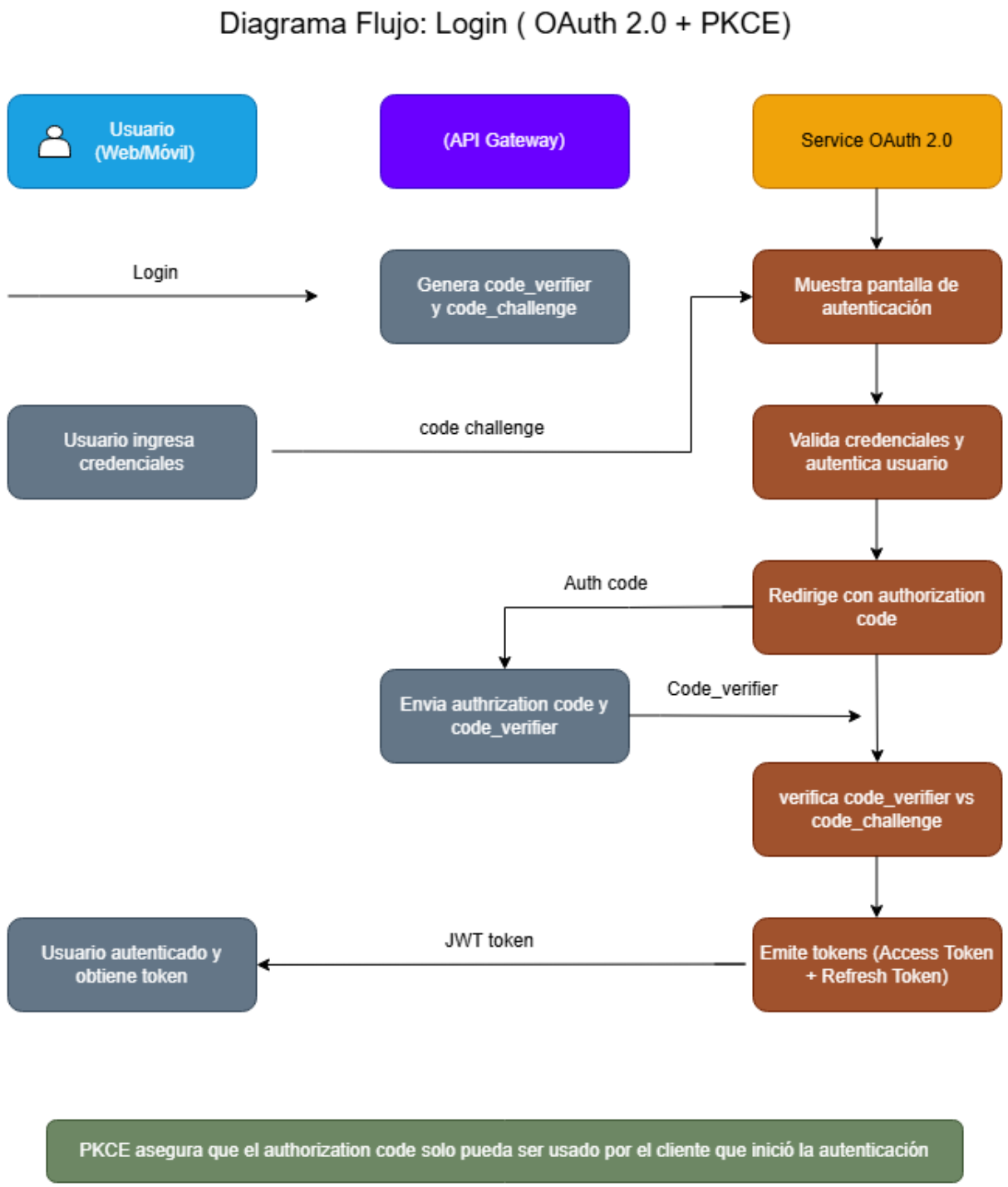
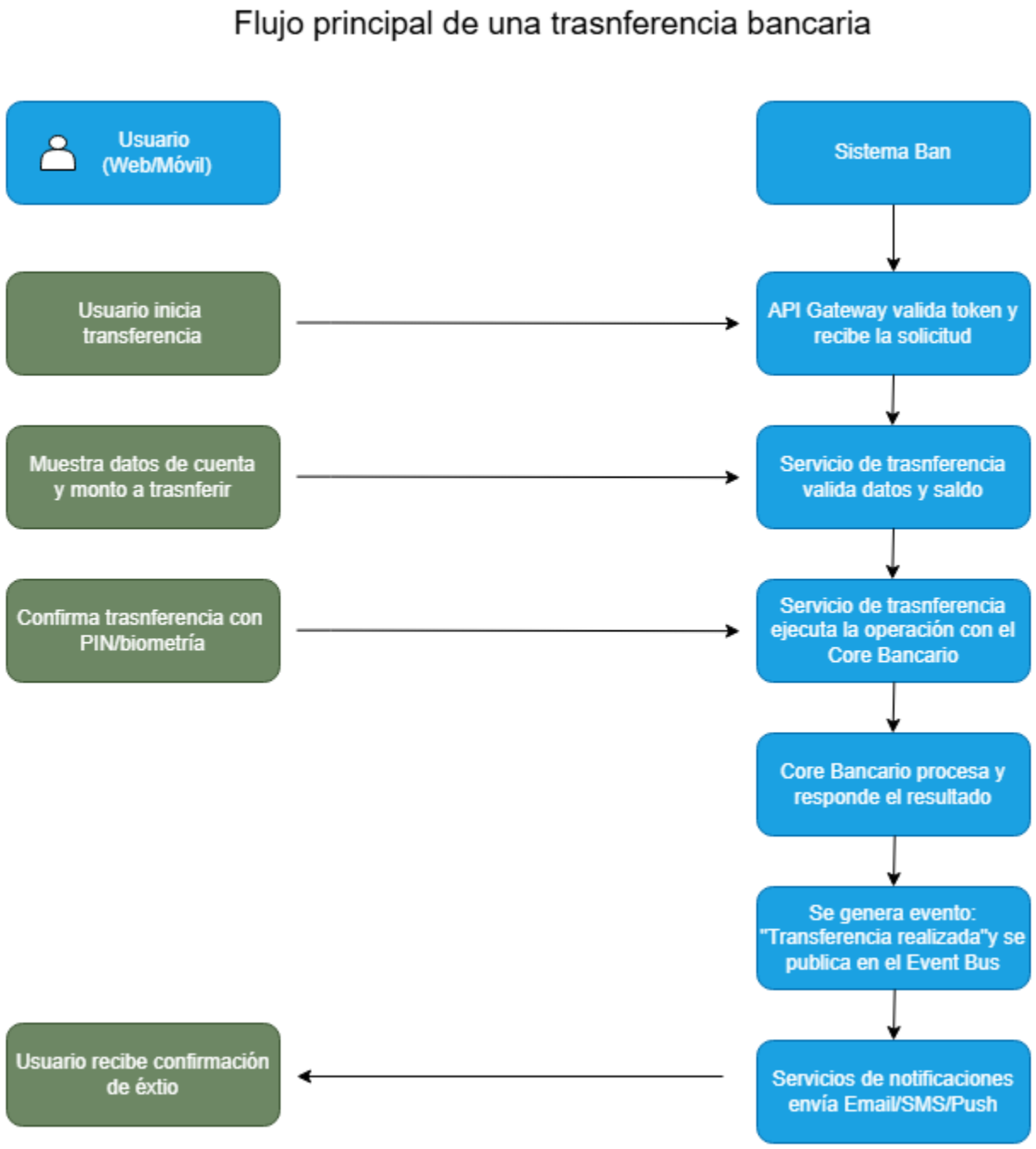



Diagrama Flujo transferencia